

```

132
133 void handleCalculatorTouch(int x, int y) {
134     int keyWidth = 35, keyHeight = 35;
135
136     // Check which calculator button is touched
137     int col = (x - 10) / (keyWidth + 5);
138     int row = (y - 80) / (keyHeight + 5);
139
140     if (col >= 0 && col < 4 && row >= 0 && row < 4) {
141         int index = row * 4 + col;
142         String calcKeys[16] = {"1", "2", "3", "+", "4", "5", "6", "-", "7", "8", "9", "*", "C", "0", "=", "/"};
143
144         String keyPressed = calcKeys[index];
145
146         if (keyPressed == "=") {
147             // Calculate result
148             text = calculateResult(text);
149         } else if (keyPressed == "C") {
150             text = "";
151         } else {
152             text += keyPressed;
153         }
154
155         // Update the result display area
156         tft.fillRect(10, 30, 240, 40, TFT_GREEN);
157         tft.setCursor(15, 50);
158         tft.print(text + "_");
159     }
160 }

```

Fig. 6: Snippet of the calculator application running under RTOS



Fig. 7: IndusPhone home screen showing calculator, call-making, incoming calls

in SMD format, is also used here. Both the mic and speaker are connected to the GSM module. The IndusBoard sends the AT command for attending or making calls, sending or receiving messages, while the 4G LTE GSM module operates independently to process these functions.

As in the earlier design, functions and a UI have been developed for various phone features, such as the dial screen, call-dialing function, incoming call screen function, calculator function, and home screen function. Now, all of these functions are integrated into a single code, which is scheduled and managed by the RTOS. The RTOS

switches each function and runs them in threads. In real time, the phone checks for incoming calls, battery percentage, touch data on the UI, network connectivity, and

EFY Note
The source code
of this project
is available
for free download at
www.electronicsforu.com

new messages, all while running the current screen's main function.

Fig. 4 shows a snippet of the home screen function, Fig. 5 depicts the contacts function, and Fig. 6 demonstrates the calculator application running under RTOS.

Construction and testing

The complete set of functions is now combined into one RTOS program. Upon powering the device, the system displays battery status, signal strength, and time on the screen. Double-tapping on the display opens the app screen where options

such as the calculator, dial pad, and contacts are available. When a call is made, the buzzer sounds until the call is either answered or disconnected. The phone interface switches to the incoming call UI. Fig. 7 shows the final IndusPhone home screen displaying the calculator, call-making, and incoming call functionality. **EFY**

Bonus. You can watch the video of the tutorial of this DIY project at <https://www.electronicsforu.com/videos-slideshows/making-your-own-programmable-indusphone-design-part-3>

Ashwini Kumar Sinha, an IoT and AI enthusiast, is Tech Journalist at EFY.